

李乐岩

leyanpower@gmail.com / 3073751449@qq.com | +86 18805506898 | GitHub: <https://github.com/lionlepower>

教育背景

爱丁堡大学 **University of Edinburgh** 英国 | 2025.09 – 2026.09 预计

高性能计算与数据科学 硕士

利物浦大学 **University of Liverpool** 英国 | 2023.09 – 2025.06

计算机科学 本科, 西交利物浦大学 2+2 项目, 一等荣誉学位

技术能力

编程语言: C/C++、Python, 具备数值计算程序、并程序、性能测试代码和自动化分析脚本开发经验; 掌握 SQL 基础, 能够完成实验数据建模、查询与分析; 了解 JavaScript、HTML、CSS, 可配合完成基础 Web 展示与系统原型开发。

高性能计算: MPI、OpenMP、PETSc、Slurm, 具备在 **ARCHER2** 超算平台上进行大规模并行实验、作业调度、日志分析和性能评估的经验; 理解 MPI 进程通信、OpenMP 线程并行、hybrid MPI+OpenMP、strong scaling、weak scaling、speedup、parallel efficiency 等概念。

性能分析与系统基础: 理解进程/线程、同步开销、通信开销、I/O、Cache locality、NUMA、memory bandwidth、负载均衡, 可从计算、通信、内存和线程调度角度解释性能瓶颈

数据处理与可视化: 熟悉 Pandas、NumPy、Matplotlib, 可完成实验日志解析、CSV 数据整理、runtime / speedup / parallel efficiency 等指标计算, 以及性能曲线、扩展性图表和对比报告生成。

AI 与 Agent 应用: 熟悉 PyTorch, 具备深度学习和多智能体强化学习实验经验; 了解 RAG、向量检索、embedding、FAISS / Chroma、LangChain / LlamaIndex、Agent tool calling、prompt engineering 和 Markdown 报告生成等 AI 应用开发流程。

后端与工程工具: 了解 FastAPI、MySQL / SQLite、Redis、REST API、HTTP、TCP、UDP 等后端与网络基础; 理解索引、事务、锁、查询优化、缓存 TTL、缓存穿透、缓存击穿、日志追踪和 TraceID 等工程概念; 熟悉 Linux、Bash、Git、VS Code、Docker 基础操作。

项目经历

面向 PETSc 并行求解器的 **HPC Benchmark** 与性能分析系统

GitHub: <https://github.com/lionlepower/petsc-benchmark-suite>

项目描述: 该项目围绕 PETSc 线性求解器的并行性能评估展开, 构建可复现的 HPC benchmark 实验流程, 用于比较 MPI-only 与 hybrid MPI+OpenMP 配置下的性能表现, 并在 ARCHER2 超算平台上完成大规模实验验证。项目覆盖 Slurm 作业调度、PETSc 参数配置、MPI / OpenMP 并行实验、日志解析、指标计算、性能可视化和 Linaro MAP profiling 分析, 重点研究不同问题规模、核心数、MPI ranks 与 OpenMP threads 对求解性能的影响。

技术栈: C/C++、Python、PETSc、MPI、OpenMP、Slurm、Bash、Pandas、NumPy、Matplotlib、Linux、ARCHER2、Linaro MAP

工作内容:

- 围绕 PETSc KSP 线性求解器构建可复现 benchmark 流程, 覆盖编译、参数生成、Slurm 提交、日志收集、结果解析、指标计算、可视化和 profiling 分析
- 在 ARCHER2 上完成 2D / 3D stencil 基准实验, 当前覆盖约 1.00M、5.02M、9.99M、20.25M、40.96M unknowns; 最大已完成规模为 40,960,000 unknowns, 实验核心数覆盖至 128 cores.
- 设计 nodes、MPI ranks、OpenMP threads 组合实验, 对比 MPI-only 与 hybrid MPI+OpenMP, 并保留 ranks / threads / total cores / KSP iterations / solver status / error logs 等字段用于复盘
- 使用 Python 编写日志解析与数据清洗脚本, 从原始 Slurm / PETSc 运行输出中提取 problem size、nodes、ranks、threads、runtime、solver status、KSP 迭代信息和异常日志等关键字段。
- 使用 Pandas 和 Matplotlib 对实验结果进行结构化处理, 计算 runtime、speedup、parallel efficiency 等指标, 并生成不同问题规模和核心数下的扩展性对比图表。
- 使用 Linaro MAP 辅助分析程序运行过程中的性能瓶颈, 对比 MPI-only 与 hybrid MPI+OpenMP 执行模式, 观察计算时间、MPI 通信时间、OpenMP 占比、线程利用率和潜在负载不均衡问题, 评估 hybrid execution 在不同问题规模下的适用场景。
- 结合 strong scaling、Amdahl's Law、进程 / 线程模型、Cache locality、NUMA 和 memory bandwidth 等基础概念分析实验结果, 定位并行效率下降的可能原因。
- 整理实验文档、运行说明和结果分析报告, 使 benchmark 流程具备可复现性, 并支持后续扩展到更多 PETSc

面向 HPC Benchmark 的 AI Agent 智能性能分析系统

项目描述：基于 PETSc benchmark 产生的实验日志、性能指标和分析文档，构建一个面向高性能计算实验场景的 AI Agent 性能分析系统。系统支持导入 Slurm / PETSc 运行日志，解析并存储实验结果，查询不同并行配置下的性能表现，并结合 RAG 检索、Agent 工具调用、数据库查询、缓存优化和 Markdown 报告生成，辅助分析 MPI-only 与 hybrid MPI+OpenMP 配置下的性能差异。项目目标是将 HPC benchmark 数据、实验文档和智能问答流程结合起来，形成一个可扩展的 AI Infrastructure 分析工具

技术栈：Python、FastAPI、SQLite / MySQL、Redis、Pandas、Matplotlib、FAISS / Chroma、LangChain / LlamaIndex、Markdown、REST API、Linux、Git、Docker

工作内容：

- 设计 AI Agent 性能分析系统整体架构，将系统拆分为 API 接入层、实验数据层、日志解析层、RAG 检索层、Agent 工具调用层、缓存层、报告生成层和可观测性模块。
- 使用 FastAPI 构建后端接口，支持实验日志导入、自然语言查询、历史结果检索、性能指标计算、Markdown 报告生成和健康检查等功能，并统一处理参数校验、异常返回和接口响应格式。
- 编写 Slurm / PETSc 日志解析模块，从原始输出中提取 problem size、nodes、MPI ranks、OpenMP threads、total cores、runtime、execution mode、solver status 和 error message 等字段，并转换为结构化实验数据。
- 设计实验结果数据库表结构，存储 benchmark 配置、运行状态、性能指标、错误信息和报告记录，并支持按 problem size、execution mode、total cores、created_at 等字段查询历史实验结果。
- 针对常见查询场景设计索引策略，例如按问题规模和核心数查询最优配置、按执行模式对比 MPI-only 与 hybrid 结果、按时间范围追踪实验记录，为后续 MySQL 查询优化和慢查询分析预留空间。
- 使用 Pandas 实现性能指标计算模块，支持自动计算 runtime、speedup、parallel efficiency，并按 problem size、nodes、ranks、threads、execution mode 生成分组对比结果。
- 构建 RAG 检索模块，将 PETSc、MPI、OpenMP、Slurm、Linaro MAP 和个人实验笔记纳入知识库，完成 Markdown 文档切分、embedding 生成、向量索引构建和 top-k 语义检索。
- 设计 Agent tool calling 流程，使系统能够根据用户问题自动选择数据库查询、日志解析、指标计算、知识库检索、图表生成和 Markdown 报告生成等工具，完成多步骤性能分析任务。
- 引入 Redis 缓存热点实验查询、常用统计结果、短期会话状态和 query hash 对应的分析结果，通过 TTL 控制缓存生命周期，减少重复数据库查询、向量检索和模型调用开销。
- 为每次分析请求生成 TraceID，记录用户问题、工具调用链路、缓存命中情况、数据库查询耗时、RAG 检索结果和报告生成耗时，便于定位检索失败、工具调用错误和性能瓶颈。
- 针对 Agent 应用中的常见问题进行优化，包括上下文过长、检索结果不准确、重复生成、缓存数据过期、工具调用失败和性能分析结论不可解释等问题。
- 输出适配 Obsidian / GitHub Markdown 的性能分析报告，支持自动生成实验摘要、配置对比表、扩展性分析、瓶颈解释和后续调优建议。

论文发表

Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis.
International Conference on Engineering Management, Information Technology and Intelligence, EMITI 2024.